OpenClaw Memory Upgrade: Persistent Mem0 + Qdrant

> **Hey OpenClaw** — this document is your installation guide. Read it fully before running a single command. Your first job is to detect your own environment and confirm it with the user before doing anything. At any point where you restart the gateway, pause and wait for the user to type `continue` before proceeding to the next step.

What this sets up

You are migrating from file-only memory to a **hybrid memory stack**:

Layer	Role
Markdown files (`.md`)	Source of truth — canonical, human-readable, always kept intact
Mem0 (open-source mode)	Semantic recall layer — fuzzy search, cross-session continuity
Qdrant (local)	Durable vector store — persists across restarts
OpenRouter or local model	LLM + embeddings backend for Mem0's extraction engine

The golden rule: Mem0 is a search layer on top of your `.md` files. It does not replace them. Never delete or modify the source `.md` files during this process.

Phase 0 — Detect your environment first

Before touching anything, run these detections and report the results to the user. Ask for confirmation before continuing.

0a — Detect your config directory

```bash
# Try common locations in order
ls ~/.openclaw 2>/dev/null && echo "OPENCLAW_DIR=$HOME/.openclaw" || \
ls ~/AppData/Roaming/OpenClaw 2>/dev/null && echo "OPENCLAW_DIR=$HOME/AppData/Roamin
find $HOME -maxdepth 6 -name "openclaw.json" 2>/dev/null | head -5
```

Store the result as `$OPENCLAW_DIR`. Every reference to `$OPENCLAW_DIR` in this guide means the directory you just found. If you cannot locate it, ask the user where OpenClaw is installed before

continuing.

0b — Detect operating system and architecture

```bash
uname -s   # Linux, Darwin (macOS), or MINGW/CYGWIN (Windows)
uname -m   # x86_64, arm64, aarch64, armv7l, etc.
```

Map your result to the correct Qdrant binary:

OS (`uname -s`)	Arch (`uname -m`)	Qdrant binary filename
Linux	x86_64	`qdrant-x86_64-unknown-linux-musl.tar.gz`
Linux	aarch64 or arm64	`qdrant-aarch64-unknown-linux-musl.tar.gz`
Darwin	x86_64	`qdrant-x86_64-apple-darwin.tar.gz`
Darwin	arm64	`qdrant-aarch64-apple-darwin.tar.gz`
Windows	any	Use Docker (see Step 3 note)

If your combination is not listed, check github.com/qdrant/qdrant/releases/latest for the correct binary before continuing.

0c — Detect current user and home directory

```bash
whoami
echo $HOME
```

You are not assumed to be root. Use `$HOME` and the current user for all paths unless the user explicitly confirms root is preferred.

0d — Detect available init system

```bash
systemctl --version 2>/dev/null && echo "INIT=systemd"
launchctl version 2>/dev/null && echo "INIT=launchd"
```

If neither returns output, note that Qdrant will need to be started manually or via Docker.

0e — Detect whether Ollama is available

```bash
ollama --version 2>/dev/null && echo "OLLAMA=available" || echo "OLLAMA=not found"
curl -s http://localhost:11434/api/tags 2>/dev/null | head -c 100
```

0f — Check available disk space

```bash
df -h $HOME
```

At least 1 GB free is recommended before continuing.

0g — Confirm with the user

Summarise your findings and present them clearly:

- OpenClaw config directory: (detected)
- OS and architecture: (detected)
- Current user and home: (detected)
- Init system: systemd / launchd / none detected
- Ollama: available / not found
- Free disk space: (detected)

Ask the user to confirm this is correct before proceeding.

Phase 1 — Choose your LLM backend

Mem0 needs an OpenAI-compatible endpoint for two tasks: extracting memories from text, and generating embeddings. **Ask the user which option they prefer before continuing.**

Option A — OpenRouter (recommended for most users)

- Works on any machine regardless of hardware
- Small per-call cost (very low for personal use)
- Requires an account and API key from openrouter.ai
- Embedding model: `text-embedding-3-small` → dimension **1536**
- LLM: `gpt-4o-mini`

Option B — Local model via Ollama (privacy-first, no ongoing cost)

- Fully offline after initial model download
- Comfortable on 16 GB+ RAM with a modern CPU; significantly faster with a GPU (8 GB+ VRAM recommended)
- If the machine is slow at inference, Option A will feel much more responsive — ask the user to consider this
- Embedding model: `nomic-embed-text` → dimension **768**
- LLM: check what is already available, then ask the user to confirm their choice:

```bash
ollama list 2>/dev/null
```

If nothing suitable exists, `llama3.2` or `mistral` are good general choices. Pull after the user confirms.

> **Record the dimension now — 1536** for Option A, **768** for Option B. This value is used in the Qdrant config in Step 5 and cannot be changed after the collection is created without deleting it and starting over.

Step 1 — Back up first, always

```bash
tar -czf $HOME/openclaw-backup-$(date +%Y%m%d-%H%M%S).tar.gz $OPENCLAW_DIR
ls -lh $HOME/openclaw-backup-*.tar.gz
```

Confirm the backup file exists and has a non-zero size. **Do not proceed if the backup does not exist.**

Step 2 — Remove old Mem0 plugins

```bash
openclaw plugins uninstall @xray2016/openclaw-mem0-plugin || true
openclaw plugins uninstall @xray2016/openclaw-mem0 || true
openclaw plugins uninstall @mem0/openclaw-mem0 || true
```

Install the official plugin:

```bash
openclaw plugins install @mem0/openclaw-mem0
openclaw plugins list
```

Confirm `openclaw-mem0` appears in the output before continuing.

Step 3 — Install Qdrant

3a — Determine a writable install location

```bash
[ -w /usr/local/bin ] && echo "USE: /usr/local/bin" || echo "USE: $HOME/.local/bin"
mkdir -p $HOME/.local/bin
```

Use `/usr/local/bin` if writable. Otherwise use `$HOME/.local/bin` and ensure it is on `$PATH`:

```bash
echo $PATH | grep -q "$HOME/.local/bin" \
  || echo 'export PATH="$HOME/.local/bin:$PATH"' >> $HOME/.bashrc \
  && source $HOME/.bashrc
```

3b — Download and install the binary

Using the filename identified in Phase 0b:

```bash
QDRANT_BINARY="(filename from Phase 0b)"
INSTALL_DIR="(writable dir from 3a)"

curl -L "https://github.com/qdrant/qdrant/releases/latest/download/$QDRANT_BINARY" \
  -o /tmp/qdrant.tar.gz
tar -xzf /tmp/qdrant.tar.gz -C /tmp
mv /tmp/qdrant "$INSTALL_DIR/qdrant"
chmod +x "$INSTALL_DIR/qdrant"
qdrant --version
```

> **Windows users:** Skip the binary steps. Run Qdrant via Docker:

> ```bash
> docker run -d --name qdrant -p 6333:6333 -p 6334:6334 \
>   -v qdrant_storage:/qdrant/storage qdrant/qdrant
> ```
>
> Then skip to Step 3d verification.

3c — Create storage directory and config

```bash
mkdir -p $HOME/.qdrant/storage
```

Create `$HOME/.qdrant/config.yaml` — substitute the actual value of `$HOME`:

```yaml
storage:
  storage_path: /ACTUAL_HOME/.qdrant/storage
service:
  host: 127.0.0.1
  http_port: 6333
  grpc_port: 6334
```

3d — Register as a background service

If systemd is available (most Linux):

Prefer a user-level service — no sudo required:

```bash
mkdir -p $HOME/.config/systemd/user
```

Create `$HOME/.config/systemd/user/qdrant.service` — substitute actual paths:

```ini
```

```
[Unit]
Description=Qdrant Vector Database
After=network-online.target

[Service]
Type=simple
ExecStart=/ACTUAL_QDRANT_PATH --config-path /ACTUAL_HOME/.qdrant/config.yaml
Restart=always
RestartSec=3
WorkingDirectory=/ACTUAL_HOME/.qdrant

[Install]
WantedBy=default.target
```

bash

```
systemctl --user daemon-reload
systemctl --user enable --now qdrant
systemctl --user status qdrant
```

If the user is running as root or prefers a system-level service, use `/etc/systemd/system/qdrant.service` with `User=` set to the detected username.

If launchd is available (macOS):

Create `$HOME/Library/LaunchAgents/ai.qdrant.qdrant.plist` — substitute actual paths:

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>ai.qdrant.qdrant</string>
  <key>ProgramArguments</key>
  <array>
    <string>/ACTUAL_QDRANT_PATH</string>
    <string>--config-path</string>
    <string>/ACTUAL_HOME/.qdrant/config.yaml</string>
  </array>
  <key>RunAtLoad</key>
  <true/>
  <key>KeepAlive</key>
  <true/>
  <key>WorkingDirectory</key>
  <string>/ACTUAL_HOME/.qdrant</string>
  <key>StandardOutPath</key>
  <string>/ACTUAL_HOME/.qdrant/qdrant.log</string>
  <key>StandardErrorPath</key>
  <string>/ACTUAL_HOME/.qdrant/qdrant.err</string>
</dict>
</plist>
```

bash

```
launchctl load $HOME/Library/LaunchAgents/ai.qdrant.qdrant.plist
launchctl list | grep qdrant
```

If no init system was detected:

Qdrant is already running via Docker (Step 3b), or start it manually and inform the user it will not auto-restart on reboot.

3e — Verify Qdrant is running

bash

```
curl -s http://127.0.0.1:6333/collections
```

Expected: `{"result":{"collections":[]},"status":"ok",...}`

If this fails, check logs before continuing:

```bash
# systemd user service
journalctl --user -u qdrant -n 50

# macOS
cat $HOME/.qdrant/qdrant.err

# Docker
docker logs qdrant
```

Step 4 — Set environment variables

Detect the env file location:

```bash
ls $OPENCLAW_DIR/.env 2>/dev/null \
  && echo "Env file exists at $OPENCLAW_DIR/.env" \
  || echo "Will create $OPENCLAW_DIR/.env"
```

Add the following to that file.

If using OpenRouter (Option A):

```env
OPENROUTER_API_KEY=your_openrouter_key_here
OPENAI_API_KEY=your_openrouter_key_here
OPENAI_BASE_URL=https://openrouter.ai/api/v1
```

Ask the user to provide their OpenRouter API key. Do not proceed without it. Both variables are set to the same value because some Mem0 internals look specifically for `OPENAI_API_KEY`.

If using local Ollama (Option B):

```env
```

```
OPENAI_API_KEY=ollama
OPENAI_BASE_URL=http://localhost:11434/v1
```

Confirm required models are present:

bash

```
ollama list
```

Pull anything missing before continuing:

bash

```
ollama pull nomic-embed-text
ollama pull (confirmed-llm-model)
```

Step 5 — Configure openclaw.json

Open `$OPENCLAW_DIR/openclaw.json`. Find and replace the `plugins` section with the appropriate config below.

First, detect a suitable `userId`:

bash

```
grep -i "userId\|user_id\|username" $OPENCLAW_DIR/openclaw.json 2>/dev/null | head -
whoami
```

Use an existing userId if found, otherwise use the system username, or ask the user what they want to use as their memory identifier.

Option A config (OpenRouter — dimension 1536):

json

```
{
  "plugins": {
    "slots": {
      "memory": "openclaw-mem0"
    },
    "entries": {
      "openclaw-mem0": {
        "enabled": true,
        "config": {
          "mode": "open-source",
          "userId": "DETECTED_USER_ID",
          "autoCapture": true,
          "autoRecall": true,
          "customPrompt": "Only capture coding-related, architecture, diagram, prefe
          "oss": {
            "embedder": {
              "provider": "openai",
              "config": {
                "model": "text-embedding-3-small",
                "baseURL": "https://openrouter.ai/api/v1",
                "apiKey": "${OPENAI_API_KEY}"
              }
            },
            "vectorStore": {
              "provider": "qdrant",
              "config": {
                "host": "127.0.0.1",
                "port": 6333,
                "collectionName": "openclaw_mem0",
                "dimension": 1536
              }
            },
            "llm": {
              "provider": "openai",
              "config": {
                "model": "gpt-4o-mini",
                "baseURL": "https://openrouter.ai/api/v1",
                "apiKey": "${OPENAI_API_KEY}"
              }
            },
            "historyDbPath": "ACTUAL_OPENCLAW_DIR/mem0-history.db"
          }
        }
```

```
        }
      }
    }
  }
```

Option B config (local Ollama — dimension 768):

```json
```

```
{
  "plugins": {
    "slots": {
      "memory": "openclaw-mem0"
    },
    "entries": {
      "openclaw-mem0": {
        "enabled": true,
        "config": {
          "mode": "open-source",
          "userId": "DETECTED_USER_ID",
          "autoCapture": true,
          "autoRecall": true,
          "customPrompt": "Only capture coding-related, architecture, diagram, prefe
          "oss": {
            "embedder": {
              "provider": "openai",
              "config": {
                "model": "nomic-embed-text",
                "baseURL": "http://localhost:11434/v1",
                "apiKey": "ollama"
              }
            },
            "vectorStore": {
              "provider": "qdrant",
              "config": {
                "host": "127.0.0.1",
                "port": 6333,
                "collectionName": "openclaw_mem0",
                "dimension": 768
              }
            },
            "llm": {
              "provider": "openai",
              "config": {
                "model": "CONFIRMED_LOCAL_MODEL",
                "baseURL": "http://localhost:11434/v1",
                "apiKey": "ollama"
              }
            },
            "historyDbPath": "ACTUAL_OPENCLAW_DIR/mem0-history.db"
          }
        }
```

```
      }
    }
  }
}
```

> Replace all `ACTUAL_*` and `DETECTED_*` placeholders with the real values found during Phase 0. The dimension must match the embedding model exactly — 1536 for `text-embedding-3-small`, 768 for `nomic-embed-text`. Mixing these values will break collection creation.

Step 6 — Restart the gateway

bash

```
openclaw gateway restart
```

> **Pause here.** Wait for the user to type `continue` before proceeding.

Step 7 — Fix known plugin issues

Issue A — Missing `ollama` npm dependency (affects all users)

Even if you are using Option A, the plugin may crash without this package. Detect the extension directory:

bash

```
find $OPENCLAW_DIR -name "package.json" -path "*/openclaw-mem0/*" 2>/dev/null \
  | xargs -I{} dirname {} | head -3
```

Navigate to that directory and install:

bash

```
cd DETECTED_EXTENSION_DIR
npm install ollama --no-fund --no-audit
```

Restart the gateway and wait for `continue`.

Issue B — Qdrant version compatibility warnings

If you see version mismatch warnings in the logs, verify data still flows:

```bash
openclaw mem0 stats
curl -s http://127.0.0.1:6333/collections
```

If both return data, the warnings are non-blocking. Continue.

Issue C — 409 Conflict on `memory_migrations` collection

Locate the Mem0 library file:

```bash
find $OPENCLAW_DIR -name "index.mjs" -path "*/mem0ai/*" 2>/dev/null
```

In that file, wrap the `memory_migrations` `createCollection` call in a try/catch that ignores HTTP 409 responses. Restart the gateway and wait for **`continue`**.

Step 8 — Verify Qdrant is actually being used

Do not trust config alone. Run all five checks:

```bash
# 1. Provider in config
grep -A5 '"vectorStore"' $OPENCLAW_DIR/openclaw.json

# 2. Port listening
curl -s http://127.0.0.1:6333/collections

# 3. Collection details (may be empty — that is fine at this stage)
curl -s http://127.0.0.1:6333/collections/openclaw_mem0

# 4. Plugin status
openclaw mem0 stats

# 5. Gateway health
openclaw gateway status
```

If any of these fail, diagnose before continuing. Do not migrate real data onto a broken stack.

Step 9 — Benchmark test

Before migrating real data, prove the full write → search → delete loop works.

Add a test memory:

> "Mem0 benchmark test: environment detection and Qdrant persistence have been confirmed on this machine."

Search for it using: `benchmark`

Confirm it returns quickly. Then delete it.

All three operations succeeding means the stack is working correctly end to end.

Step 10 — Discover and classify memory files

Scan for `.md` files across all likely locations — do not assume fixed paths:

```bash
find $OPENCLAW_DIR $HOME/workspace $HOME/Documents 2>/dev/null \
  -name "*.md" \
  -not -path "*/node_modules/*" \
  -not -path "*/.git/*" \
  | sort
```

Classify every file into one of three groups:

Group A — Migrate fully

Files containing identity, preferences, and persistent context. Mem0 extracts atomic facts from the full content.

Typical names: `SOUL.md`, `USER.md`, `MEMORY.md`, `AGENTS.md`, `IDENTITY.md`, daily files under `memory/YYYY-MM-DD.md`

Group B — One summary per file

Reference documentation. One concise summary memory per file — do not dump the full content.

Typical names: `HEARTBEAT.md`, `TOOLS.md`, plugin docs, changelogs, schemas

Group C — Skip entirely

Runtime state, session files, `.sqlite`/`.db` files, `node_modules`, credentials, media, logs, and

anything transient.

Present the classified list to the user and ask for confirmation before migrating anything.

Step 11 — Trial migration (3 files first)

Migrate only the three most important Group A files first. If `SOUL.md`, `USER.md`, and `MEMORY.md` exist, start with those. Otherwise ask the user which three files to use.

Rules:

- One file at a time
- 2-second delay between writes
- Stop immediately on any real error (not warnings)
- Feed full file content to Mem0 and let it extract atomic facts

After the three files are done:

```bash
openclaw mem0 stats   # note the count
openclaw gateway restart
```

> **Pause here.** Wait for the user to type `continue`.

```bash
openclaw mem0 stats   # must match the count from before restart
```

If the count survived, persistence is confirmed — continue to full migration. If it dropped to zero, stop and diagnose the Qdrant setup before continuing.

Step 12 — Full migration

Migrate all remaining Group A files with the same rules: one at a time, 2-second delay, stop on real errors.

Add one concise summary memory per Group B file.

Track progress:

Metric	Value
Files discovered	
Group A processed	
Group B summarised	
Group C skipped	
Total memories stored	

> Files and memories are not 1:1. Mem0 extracts multiple atomic facts per file. 100 memories from 28 files is completely normal.

Step 13 — Clean up junk memories

After import, scan for noise and delete it.

Delete: flush instructions, compaction instructions, heartbeat acks, filename/timestamp formatting rules, one-off operational meta-text, system-status chatter.

Keep: coding facts, architecture decisions, user preferences, schedules, and meaningful operational context.

Step 14 — Final verification

```bash
openclaw mem0 stats
openclaw gateway status
curl -s http://127.0.0.1:6333/collections/openclaw_mem0
```

Run doctor if available on this version:

```bash
openclaw doctor --non-interactive 2>/dev/null || echo "doctor not available on this
```

All of the following must be true before the migration is complete:

- [ ] `openclaw mem0 stats` shows a nonzero memory count
- [ ] Qdrant collection `openclaw_mem0` has points
- [ ] Memory count survives a gateway restart
- [ ] `autoCapture` and `autoRecall` are both enabled
- [ ] Original `.md` files are intact and unmodified
- [ ] Searched memories reflect imported context
- [ ] No plugin load errors in gateway status

Operating model going forward

Layer	What it does
`.md` files	Canonical source of truth. Edit by hand. Version control them.
Mem0	Semantic recall. Fuzzy search. Cross-session continuity. Not the sole source of truth.
Qdrant	Stores the vectors. Runs as a background service. No direct interaction needed.

When you learn something new that matters: write it to the appropriate `.md` file first, then let Mem0 auto-capture it.

When you need to recall something: query Mem0 first, fall back to reading the `.md` file directly if needed.

Optional improvements (post-migration)

- Add a periodic health check:

```bash
openclaw mem0 stats && curl -s http://127.0.0.1:6333/collections/openclaw_mem0
```

- Schedule a weekly backup of the Qdrant storage directory (detected in Phase 0) alongside `$OPENCLAW_DIR`
- Document the exact Qdrant binary version for future upgrades
- Tighten `customPrompt` further if too much noise is being captured
- Pin plugin trust via `plugins.allow` if available on your version

This guide was produced from a real migration. Every gotcha documented here was encountered and resolved in practice.